

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO TUẦN SÁU
MÔN THỰC TẬP CƠ SỞ

Giảng viên hướng dẫn : Kim Ngọc Bách

Sinh viên thực hiện : Nguyễn Trung Nghĩa

Mã Sinh Viên : B23DCCN602

Hà Nội - 2026

1. Mục tiêu hướng đến

Tiếp nối thành công của tuần thứ năm trong việc hoàn thiện hệ thống lõi giám sát AI và lưu trữ đám mây, tuần thứ sáu của dự án tập trung vào việc biến hệ thống từ một bộ API mở thành một nền tảng phần mềm quản trị an ninh khép kín, an toàn và chuyên nghiệp.

Các mục tiêu chính trong tuần này bao gồm:

- Xây dựng hệ thống Xác thực và Phân quyền người dùng.
- Áp dụng kiến trúc lưu trữ đa mô hình kết hợp SQL và NoSQL.
- Chuẩn hóa đồng nhất 100% cấu trúc dữ liệu phản hồi cho toàn bộ API.
- Phát triển module Báo cáo và Thống kê sử dụng truy vấn nâng cao.
- Xây dựng tính năng kết xuất dữ liệu (Export Excel) tối ưu tài nguyên máy chủ.

Việc hoàn thành các mục tiêu này giúp hệ thống đạt tiêu chuẩn bảo mật của một ứng dụng thực tế, sẵn sàng để tích hợp với giao diện Frontend ở giai đoạn cuối.

2. Phát triển hệ thống Xác thực và Phân quyền

Trong một hệ thống giám sát an ninh thực tế, dữ liệu camera và lịch sử vi phạm là những thông tin nhạy cảm. Do đó, việc xây dựng một bức tường lửa quản lý quyền truy cập là yêu cầu bắt buộc.

2.1. Áp dụng kiến trúc lưu trữ đa mô hình

Thay vì cố gắng nhồi nhét dữ liệu người dùng vào MongoDB hiện tại, em đã quyết định sử dụng đồng thời hai hệ quản trị cơ sở dữ liệu:

- MongoDB Atlas (NoSQL):
 - Tiếp tục đảm nhiệm việc lưu trữ lịch sử nhận diện AI.
 - Lý do: Dữ liệu AI có tính linh hoạt cao, tần suất ghi liên tục và chứa nhiều mảng tọa độ phức tạp.
- SQLite kết hợp SQLAlchemy (Relational DB):
 - Được tích hợp mới để quản lý thông tin tài khoản người dùng.
 - Lý do: Dữ liệu người dùng đòi hỏi tính toàn vẹn cao, ràng buộc quan hệ chặt chẽ và không thay đổi cấu trúc thường xuyên.

Đánh giá: Quyết định thiết kế này giúp hệ thống tận dụng tối đa thế mạnh của từng loại cơ sở dữ liệu, tối ưu hóa hiệu năng tổng thể và thể hiện tư duy thiết kế hệ thống phân

tán. Toàn bộ quá trình khởi tạo cả hai DB được gộp chung một cách an toàn vào lifespan của FastAPI.

2.2. Hệ thống Xác thực (Authentication)

Hệ thống xác thực được xây dựng dựa trên tiêu chuẩn bảo mật hiện đại:

- Mã hóa mật khẩu: Sử dụng thư viện `pwdlib` để băm (hash) mật khẩu một chiều, đảm bảo không lưu trữ mật khẩu dưới dạng văn bản thô.
- Giao thức JWT (JSON Web Token): Áp dụng thuật toán HS256. API `/auth/login` cấp phát Access Token có thời hạn, giúp hệ thống hoạt động theo cơ chế Stateless, dễ dàng mở rộng máy chủ sau này.
- Quản lý vòng đời tài khoản:
 - Xây dựng đầy đủ các luồng nghiệp vụ:
 - Đăng ký
 - Gửi email xác thực tài khoản
 - Quên mật khẩu và Đổi mật khẩu
 - Tác vụ gửi email được đưa vào `BackgroundTasks` để không làm chặn luồng phản hồi API.

2.3. Hệ thống Phân quyền (Role-Based Access Control - RBAC)

Để quản lý việc ai được làm gì, em đã bổ sung trường role vào Database người dùng và thiết lập cơ chế kiểm duyệt tại tầng Router.

- Cơ chế "Người gác cổng" (Gatekeeper): Em xây dựng lớp `RoleChecker` đóng vai trò là một Dependency trong FastAPI.
- Phân chia vai trò:
 - ADMIN: Có toàn quyền. Được phép tạo tài khoản cho nhân viên mới (`/register`), xem báo cáo và xuất file Excel.
 - GUARD (Bảo vệ): Chỉ có quyền xem luồng Livestream camera và xem danh sách lịch sử vi phạm trong ca trực.
- Lý do thiết kế: Việc chặn quyền truy cập ngay tại tầng Router thông qua `Depends()` giúp hệ thống từ chối các yêu cầu trái phép ngay lập tức, tiết kiệm tài nguyên tính toán cho tầng Service.

3. Xây dựng Module Báo cáo và Phân tích dữ liệu

Dữ liệu vi phạm nếu chỉ để xem dưới dạng danh sách sẽ mang lại ít giá trị cho cấp quản lý. Do đó, em đã phát triển thêm Router /reports để cung cấp số liệu thống kê nghiệp vụ.

3.1. Tối ưu hóa truy vấn bằng MongoDB Aggregation Pipeline

Để thống kê số lượng vi phạm theo ngày (Daily) và theo giờ (Hourly), hệ thống cần xử lý hàng ngàn bản ghi.

- Cách tiếp cận ban đầu (Chưa tối ưu): Tải toàn bộ dữ liệu từ DB về Python, sau đó dùng vòng lặp for để đếm. Cách này gây tiêu tốn lượng lớn RAM và làm chậm Server.
- Cách tiếp cận hiện tại (Tối ưu): Áp dụng Aggregation Pipeline (\$match, \$group, \$dateToString). Toàn bộ phép toán thống kê phức tạp được đẩy trực tiếp cho engine của MongoDB Atlas xử lý ở tầng Database.
- Kết quả: API trả về dữ liệu biểu đồ gần như tức thời, tiết kiệm đến 80% tài nguyên xử lý của máy chủ Backend.

3.2. Kết xuất dữ liệu (Export Excel) tối ưu tài nguyên

Chức năng xuất danh sách vi phạm ra file Excel (.xlsx) được thiết kế đặc biệt để tối ưu cho việc vận hành thực tế.

- Lý do thiết kế: Thay vì tạo file vật lý lưu tạm trên ổ cứng server (dễ gây lỗi tràn bộ nhớ, rác hệ thống và lỗi phân quyền đọc/ghi file), em sử dụng thư viện io.BytesIO.
- Cơ chế hoạt động: File Excel được sinh ra trực tiếp trên bộ nhớ RAM (In-memory) và trả về cho người dùng thông qua StreamingResponse của FastAPI. Ngay sau khi người dùng tải xong, vùng nhớ sẽ tự động được giải phóng, đảm bảo server luôn sạch sẽ và an toàn.

4. Chuẩn hóa cấu trúc phản hồi

Để tạo ra một bộ API chuẩn, thuận tiện nhất cho việc tích hợp giao diện (Frontend) sau này, em đã tiến hành chuẩn hóa mọi phản hồi từ Server.

- Xây dựng BaseResponse: Một Pydantic Model sử dụng Generic Type, bọc mọi dữ liệu trả về theo khung chuẩn: code, message, result, error_code, errors.

- Đồng bộ Exception Handlers: Cập nhật lại toàn bộ bộ xử lý lỗi toàn cục (Global Exception Handler) được viết ở tuần 5. Giờ đây, dù API thành công (200 OK) hay thất bại (400, 422, 500...), Frontend luôn nhận được cùng một cấu trúc JSON.
- Ngoại lệ có chủ đích: Duy nhất API /auth/login không bọc trong BaseResponse mà trả về cấu trúc phẳng. Lý do là để tuân thủ nghiêm ngặt tiêu chuẩn RFC 6749 (OAuth2 Password Flow), đảm bảo tương thích 100% với các công cụ kiểm thử như Swagger UI và thư viện xác thực phía Client.

5. Kết quả đạt được trong tuần 6

Sau khi hoàn thành các công việc trong tuần thứ sáu, các kết quả chính đạt được bao gồm:

- Hoàn thành tích hợp kiến trúc lưu trữ kép: SQLite cho quản trị tài khoản và MongoDB cho nhật ký AI.
- Triển khai thành công hệ thống bảo mật JWT, mã hóa mật khẩu.
- Hoàn thiện hệ thống phân quyền (RBAC) chặt chẽ giữa Admin và Guard bảo vệ tận gốc các endpoint quan trọng.
- Xây dựng thành công hệ thống API báo cáo thống kê với tốc độ cao nhờ MongoDB Aggregation.
- Tích hợp tính năng xuất báo cáo Excel dạng Streaming an toàn cho máy chủ.
- Chuẩn hóa toàn diện 100% cấu trúc phản hồi API

6. Khó khăn gặp phải và cách khắc phục

Trong tuần thứ sáu của dự án, quá trình phát triển hệ thống xác thực, phân quyền và báo cáo đã phát sinh một số thách thức đáng kể liên quan đến việc tích hợp chuẩn bảo mật OAuth2 vào kiến trúc API hiện có, chuẩn hóa thông báo lỗi validation, tối ưu tác vụ nền không đồng bộ. Các vấn đề này chủ yếu xuất phát từ yêu cầu vừa đảm bảo trải nghiệm người dùng nhất quán, vừa tuân thủ các chuẩn bảo mật quốc tế, đồng thời duy trì hiệu năng cao cho luồng xử lý bất đồng bộ. Việc giải quyết đòi hỏi sự can thiệp sâu vào cơ chế xử lý response, exception handling, và quản lý tài nguyên đa luồng.

6.1. Xung đột giữa Cấu trúc phản hồi và Tiêu chuẩn OAuth2

Khi bọc endpoint `/auth/login` vào lớp `BaseResponse`, công cụ Swagger UI không thể tự động bóc tách `access_token` nằm sâu bên trong trường `result`, dẫn đến lỗi 401 Unauthorized khi gọi các API khác cần token. Nguyên nhân là do OAuth2 yêu cầu response trả về phẳng với trường `access_token` ở cấp cao nhất.

Cách khắc phục: Em đã phân tích đặc tả OAuth2 và quyết định giữ nguyên thiết kế trả về phẳng riêng cho endpoint login, trong khi vẫn duy trì `BaseResponse` cho tất cả các API nghiệp vụ khác. Điều này vừa đảm bảo tính tương thích chuẩn quốc tế, vừa giữ được sự nhất quán cho phần còn lại của hệ thống.

6.2. Khó khăn trong việc đồng bộ hóa dữ liệu ngoại lệ

Mặc định, khi người dùng gửi dữ liệu sai định dạng (ví dụ: mật khẩu quá ngắn, email không hợp lệ), FastAPI trả về lỗi 422 với cấu trúc JSON rất phức tạp, bao gồm mảng detail chứa nhiều thông tin khó đọc đối với Frontend.

Cách khắc phục: Em đã override handler `RequestValidationError` của FastAPI. Sử dụng vòng lặp để trích xuất chính xác tên trường bị lỗi (field) và câu thông báo (message), sau đó ánh xạ chúng vào đúng định dạng `BaseResponse`. Nhờ vậy, Frontend có thể dễ dàng map lỗi vào ngay dưới từng ô input, cải thiện trải nghiệm người dùng rõ rệt.

6.3. Blocking code trong tác vụ upload ảnh lên Cloudinary

Hàm `upload_image_to_cloudinary` sử dụng SDK đồng bộ của Cloudinary, khi chạy trong `BackgroundTasks` vẫn gây nghẽn event loop, ảnh hưởng đến hiệu năng xử lý các request khác.

Cách khắc phục: Em đã chuyển hàm upload thành async và bọc lời gọi `upload()` bằng `await asyncio.to_thread()`, giúp chạy tác vụ trong thread riêng mà không block event loop. Đồng thời, cập nhật hàm `save_violation_backtask` để await kết quả upload, đảm bảo non-blocking hoàn toàn.

7. Kế hoạch thực hiện trong tuần tiếp theo

Đến thời điểm hiện tại, toàn bộ kiến trúc Backend (AI, Database, Security, Analytics) đã hoàn thiện và vận hành ổn định. Tuần tiếp theo của dự án sẽ là bước chuyển giao quan trọng sang tầng hiển thị.

7.1. Hoàn thiện API gửi email với template HTML chuyên nghiệp

Trong tuần thứ sáu, mặc dù đã xây dựng được các chức năng gửi email xác thực tài khoản và khôi phục mật khẩu. Nhưng vẫn chưa thể hoạt động do chưa hoàn thiện việc gửi email. Tuần tiếp theo, em sẽ tập trung hoàn thiện module email bằng cách xây dựng các template HTML chuyên nghiệp.

7.2. Xây dựng Frontend Dashboard

Sau khi backend đã cung cấp đầy đủ các API nghiệp vụ (xác thực, livestream, lịch sử vi phạm, báo cáo thống kê, xuất Excel), bước tiếp theo là xây dựng một Web Dashboard hoàn chỉnh để người dùng cuối có thể tương tác trực tiếp.

Trong giai đoạn này, em dự kiến phát triển dashboard sử dụng HTML5, CSS3 và JavaScript, đảm bảo giao diện hiện đại, responsive và tương thích đa trình duyệt.

7.3. Tích hợp API

Để dashboard hoạt động chính xác và an toàn, em sẽ thực hiện các công việc tích hợp sau:

- Tạo module api.js: Định nghĩa các hàm gọi fetch đến từng endpoint, tự động đính kèm JWT token từ localStorage vào header Authorization: Bearer <token>.
- Xử lý lỗi tập trung: Interceptor kiểm tra mã lỗi HTTP 401 => xóa token và chuyển về trang login; lỗi 422 => hiển thị thông báo lỗi validation dưới từng ô input theo đúng format từ backend.
- Bảo vệ route: Kiểm tra sự tồn tại của token trước khi cho phép truy cập các trang yêu cầu đăng nhập